Django REST Framework (DRF)

Complete Interview Questions & Answers Guide (Technical & Conceptual)

SECTION 1: Core Fundamentals & Architectural Concepts

Q1. What is DRF and why use it instead of standard Django?

Interview Response (English):

Standard Django follows the MTV (Model-Template-View) pattern built for server-side HTML rendering. DRF is an extension designed to create decoupled RESTful APIs. It provides built-in serializers, authentication classes, permission systems, generic views, and a web-browsable API, returning clean JSON data instead of HTML templates.

Tamil / Conceptual Meaning (Puridhal):

Normal Django HTML page render pannum (Server-Side Rendering). DRF vandhu client applications (React, Flutter, Mobile apps, Vue)-ku JSON API provide panna use aagum. Idhula Serializers, Token Authentication, Permissions ellame out-of-the-box irukku.

Q2. What is Serialization and Deserialization in DRF?

Interview Response (English):

Serialization is the process of converting complex data types like Django Model instances or QuerySets into Python native datatypes (dictionaries/lists), which can then easily be converted into JSON or XML format.
Deserialization is the reverse process: parsing incoming JSON data from request bodies, validating it against rules, and converting it back into complex Python data types or saving directly into Django models.

Tamil / Conceptual Meaning (Puridhal):

Serialization: Database records (QuerySets) -> JSON data-va maathradhu (GET request-la client-ku anuppa).
Deserialization: Frontend JSON data -> validate panni Database model-la save panradhu (POST / PUT request-la).

Q3. What is the difference between Serializer and ModelSerializer?

Interview Response (English):

Serializer is the base class where every field must be explicitly declared, and create() and update() methods must be manually implemented by the developer.
ModelSerializer is a shortcut subclass that automatically inspects a Django Model, generates the appropriate serializer fields, creates default validators (like unique constraints), and includes ready-to-use implementations of create() and update() inside its inner Meta class.

Tamil / Conceptual Meaning (Puridhal):

Serializer: Django forms.Form maadhiri. Ovvoru field and create/update logic-aiyum neengale manual-ah ezhudhanum.
ModelSerializer: Django forms.ModelForm maadhiri. 'model = Camera' and 'fields = __all__' nu kudutha model-la irukkura fields and save logic automatic-ah generate aagidum.

Q4. When and why should we use many=True in Serializers?

Interview Response (English):

When serializing a QuerySet containing multiple records (e.g. Camera.objects.filter()), you must pass many=True so the serializer handles it as an iterable list and produces a JSON Array [{ ... }, { ... }]. If serializing a single model instance (e.g.

Camera.objects.get()), omit many=True to produce a single JSON Object { ... }. Passing a single object with many=True raises an 'Object is not iterable' error.

Tamil / Conceptual Meaning (Puridhal):

Database-la filter() pottu neraya records (List/QuerySet) edukumbodhu serializer-ku 'many=True' kudukanum, appo output '[{...}, {...}]' nu Array-ah varum. Orey oru record get() panni edukumbodhu 'many=True' poda koodadhu, appo output '{ ... }' nu Single Object-ah varum.

SECTION 2: Views, URLs & Routing

Q5. Difference between APIView, GenericAPIView, and ModelViewSet?

Interview Response (English):

APIView: Base class for class-based views. Directly handles HTTP verb methods (get, post, put, delete). Offers total control with manual code.
GenericAPIView: Inherits from APIView and introduces attributes like queryset and serializer_class. Paired with Mixins (ListModelMixin, CreateModelMixin) to reduce repeated code.
ModelViewSet: Highest level of abstraction. In 3 lines of code, it provides full CRUD (list, create, retrieve, update, destroy) and hooks into DRF Routers automatically.

Tamil / Conceptual Meaning (Puridhal):

APIView: Line-by-line custom code ezhudha use aagum.
GenericAPIView: QuerySet + Serializer set panni standard patterns handle panna use aagum.
ModelViewSet: 3 lines of code-la complete CRUD endpoints automatic-ah tharum (Production-la time save panna idhu dhaan widely used).

Q6. What is the difference between request.data and request.POST?

Interview Response (English):

Django's request.POST only parses standard form data and operates solely on HTTP POST requests.
DRF's request.data handles arbitrary request payloads (JSON, form-data, multipart files), works across POST, PUT, and PATCH methods, and automatically parses requests based on the Content-Type header.

Tamil / Conceptual Meaning (Puridhal):

Django-oda request.POST form data mattum dhaan padikkum. Aana DRF-oda request.data JSON, file upload, PUT, PATCH methods ellathaiyum automatic-ah parse pannidum.

Q7. What are Routers in DRF (DefaultRouter vs SimpleRouter)?

Interview Response (English):

Routers in DRF provide a clean way of automatically mapping URLs to ViewSets. Instead of writing individual path() entries for list, create, detail, update, and delete, a Router (DefaultRouter or SimpleRouter) automatically generates standard RESTful URL patterns.
DefaultRouter includes a default root API view and format suffixes (.json), whereas SimpleRouter provides basic URL routes without the root view.

Tamil / Conceptual Meaning (Puridhal):

Routers vandhu ViewSet-ku thevaiyana CRUD URL patterns-ah automatic-ah generate pannum. Ovvoru URL path-aiyum manual-ah ezhudha thevailla. router.register('cameras', CameraViewSet) nu sonna podhum.

```
# Example DRF Router in urls.py
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register(r'cameras', CameraViewSet, basename='camera')
urlpatterns = router.urls
```

SECTION 3: Validation, SerializerMethodField & Data Control

Q8. How does Validation work in Serializers?

Interview Response (English):

DRF supports validation at three levels:
1. Field-level validation: Define method validate_<field_name>(self, value). Used to validate a single field.
2. Object-level validation: Define method validate(self, data). Used to compare multiple fields (e.g. password match or date range).
3. Custom Validators: Reusable functions passed directly into field definition arguments.

Tamil / Conceptual Meaning (Puridhal):

1. Field-level: validate_<field> method (e.g. IP address or age check).
2. Object-level: validate(self, data) method (e.g. password & confirm_password compare panna).

```
class CameraSerializer(serializers.ModelSerializer):
    def validate_ip_address(self, value):
        if not value.startswith("192.168"):
            raise serializers.ValidationError("Only internal subnet IPs allowed.")
        return value

    def validate(self, data):
        if data.get('port') and data['port'] < 0:
            raise serializers.ValidationError("Port must be positive.")
        return data
```

Q9. What is SerializerMethodField and when to use it?

Interview Response (English):

SerializerMethodField is a read-only field used to include custom calculated or dynamic data in the serialized response that does not exist directly as a database column. DRF automatically looks for a method named get_<field_name>(self, obj) to compute and return its value.

Tamil / Conceptual Meaning (Puridhal):

Database model-la andha column irukaadhu, aana python logic vechu dynamically calculate panni frontend-ku JSON-la anuppa SerializerMethodField use aagum.

```
class CameraSerializer(serializers.ModelSerializer):
    is_online_status = serializers.SerializerMethodField()

    class Meta:
        model = Camera
        fields = ['id', 'model_name', 'is_online_status']

    def get_is_online_status(self, obj):
        return "Online" if obj.is_working else "Offline"
```

SECTION 4: Authentication & Permission Systems

Q10. What is the difference between Authentication and Permission in DRF?

Interview Response (English):

Authentication identifies WHO the user is by verifying identity tokens or credentials. If valid, request.user is populated. Permission checks WHAT the authenticated user is allowed to perform. It verifies whether the requesting user has the authority to view or execute operations on a given view or object (e.g. IsAdminUser, IsAuthenticated, ReadOnly).

Tamil / Conceptual Meaning (Puridhal):

Authentication: 'Yaar indha user?' (Identity confirm panradhu).
Permission: 'Indha user-ku idhai access panna permission irukka?' (Authorization check).

Q11. Which Authentication mechanisms are commonly used in DRF?

Interview Response (English):

DRF natively includes BasicAuthentication, SessionAuthentication, and TokenAuthentication. In modern decoupled architectures (React/Mobile), JWT (JSON Web Token) using the library djangorestframework-simplejwt is standard because it is completely stateless, supports short-lived Access Tokens and long-lived Refresh Tokens, and does not require session lookups in database storage.

Tamil / Conceptual Meaning (Puridhal):

DRF default-ah Token and Session auth tharum. Aana industry standard-la React/Flutter mobile apps-ku SimpleJWT (Access Token + Refresh Token) dhaan widely used, yen na adhu completely stateless and fast.

Q12. How do you implement a Custom Permission class in DRF?

Interview Response (English):

Subclass rest_framework.permissions.BasePermission and override:
1. has_permission(self, request, view): Evaluated for the view before any action is executed.
2. has_object_permission(self, request, view, obj): Evaluated when retrieving or updating a specific object instance (e.g. checking if request.user == obj.owner).

Tamil / Conceptual Meaning (Puridhal):

BasePermission class-la irundhu inherit panni has_permission (general view level) and has_object_permission (specific row level, e.g. edit panna andha camera owner-ah nu check panna) ezhudhanum.

```
from rest_framework.permissions import BasePermission

class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True
        return obj.created_by == request.user
```

SECTION 5: Optimization, Pagination & Advanced Questions

Q13. How do you solve the N+1 Query Problem in DRF?

Interview Response (English):

When serializing models with foreign keys or relationships, accessing related fields triggers a new database query for every row, causing 1 + N queries.
Solution:
• Use select_related() for ForeignKey and OneToOne relationships (performs a SQL JOIN).
• Use prefetch_related() for ManyToMany and reverse ForeignKeys (executes a single batched query in Python).

Tamil / Conceptual Meaning (Puridhal):

100 cameras fetch pannumbodhu, ovvoru camera-oda Section record-aiyum access panna 101 queries run aagum (N+1 Problem). Idhai solve panna:
Camera.objects.select_related('section') nu pottu orey single SQL query-ah join panni optimize pannuvom.

Q14. Explain Pagination classes in DRF and how to configure them?

Interview Response (English):

DRF provides built-in pagination classes configured in settings.py or per-view:
1. PageNumberPagination: Uses page query parameters (e.g. ?page=2&page;_size=20). Most standard for web pagination.
2. LimitOffsetPagination: Uses limit & offset query parameters (e.g. ?limit=10&offset;=30). Popular for infinite scroll.
3. CursorPagination: Token-based pagination for high-velocity datasets that avoids duplicate pages during real-time writes.

Tamil / Conceptual Meaning (Puridhal):

Large data sets-ah 10-10 items-ah split panni anuppa Pagination use aagum. 'PageNumberPagination' standard web pages-ku, 'LimitOffsetPagination' or 'CursorPagination' infinite scrolling-ku use aagum.

```
# Global Pagination in settings.py
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20
}
```

Q15. What is Throttling in DRF and why is it essential?

Interview Response (English):

Throttling controls the rate of incoming requests that clients can make to your API to prevent denial-of-service, abuse, or API scraping. DRF provides AnonRateThrottle (rate limits unauthenticated IPs, e.g. '100/day') and UserRateThrottle (rate limits authenticated users, e.g. '1000/hour') configured under DEFAULT_THROTTLE_RATES in settings.py.

Tamil / Conceptual Meaning (Puridhal):
Throttling vandhu API rate-limiting. Oru minute-ku oru user 10 requests mela panna mudiyadha maadhiri block panna idhu use aagum (Server crash aagadha maadhiri protect panna).

SECTION 6: Top 5 Golden Rules for DRF Interviews

1. Mention Key Architecture Terms: Always use terms like 'Decoupled', 'Stateless', 'Serialization', 'Validation', 'HTTP Verbs', and 'RESTful standards'.

2. Explain Why DRF over Django: Highlight that traditional Django renders HTML templates on the server, while DRF delivers JSON APIs for mobile and modern frontend applications.

3. Know when to choose ViewSet vs APIView: Use ModelViewSet for rapid standard CRUD; use APIView for custom business workflows (like camera video streaming or custom third-party webhook processing).

4. Database Query Optimization: Impress the interviewer by mentioning select_related and prefetch_related to prevent N+1 query overhead in nested serializers.

5. Security & Tokens: Emphasize JWT (SimpleJWT) with Access and Refresh tokens as modern best practice over static tokens.